Einführung in die Rechnerarchitektur

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Speichern

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Sequentielle Schaltungen

Robert Wille
robert.wille@tum.de

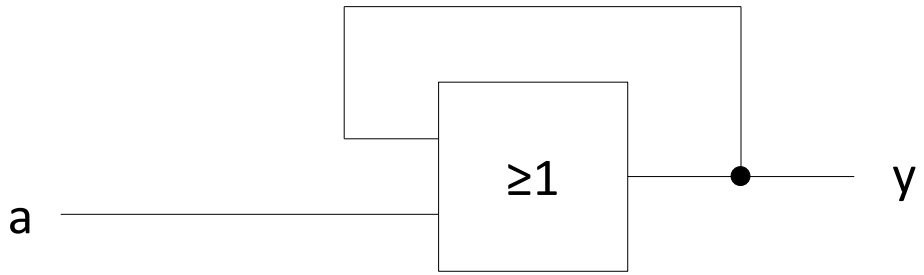
Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Rückkopplungen

- Was passiert, wenn Ausgang einer Schaltung auf Eingang zurückwirkt?



- Annahme: *a* und *y* am Anfang 0
  - Solange *a* auf 0 bleibt, keine Änderung an y
  - *a* wird 1, dann wird *y* auch 1 und bleibt so
  - Schaltung hat „Gedächtnis"

Speicherbausteine

Robert Wille
robert.wille@tum.de

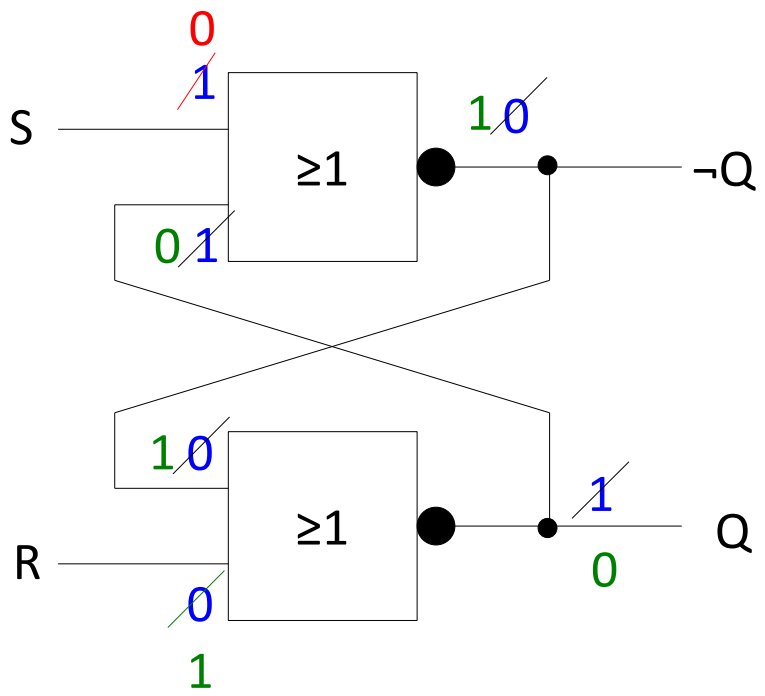
Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

RS-Flipflop

- Schaltung zum Speichern von Information
- Eingänge zum Setzen (Set) und Löschen (Reset)



Annahme: *S=1, R=0*

Annahme: *S=0, R=0*

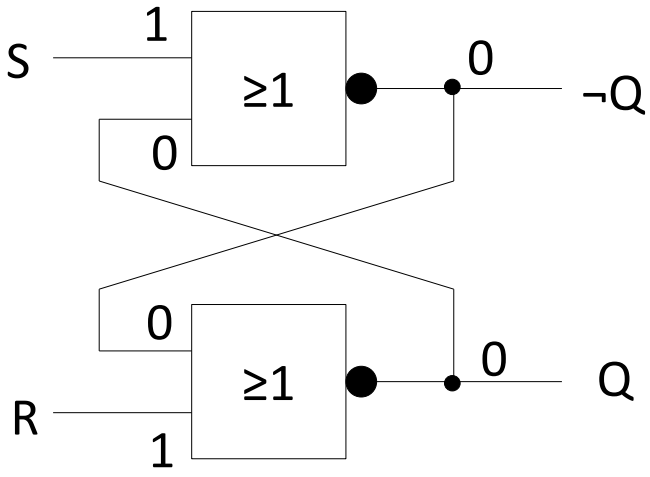
Annahme: *S=0, R=1*

RS-Flipflop: Wahrheitstabelle

R	S	Q(t+1)	¬Q(t+1)	Verhalten
0	**0**	**Q(t)**	**¬Q(t)**	**Speichern**
1	**0**	**0**	**1**	**Rücksetzen**
0	**1**	**1**	**0**	**Setzen**
1	1	?	?	?

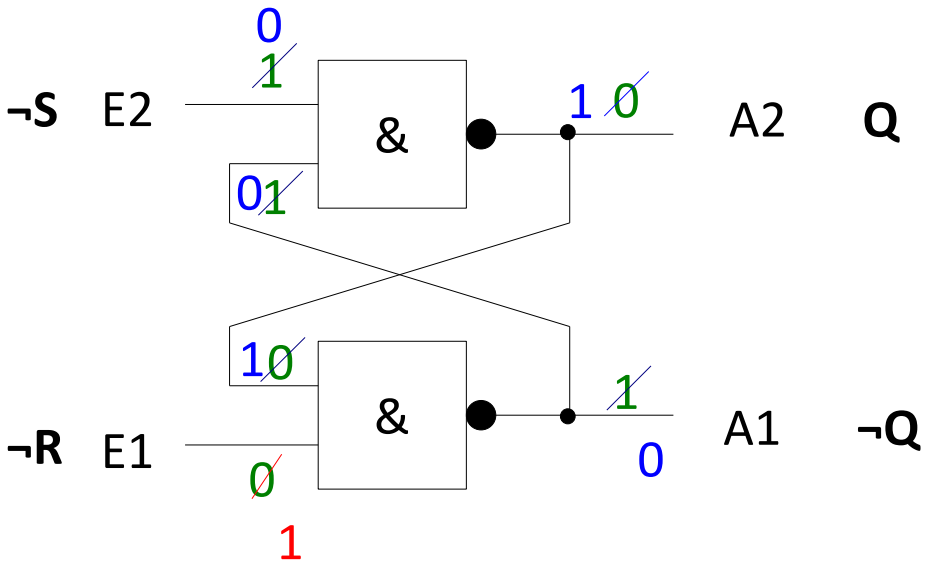
- *Q(t)* : aktueller Zustand
- *Q(t+1)* : Folgezustand
- Beide Eingänge 0 ⇒ Zustand wird gespeichert
- Was passiert bei *R=1* und *S=1*?

RS-Flipflop: Verbotener Zustand



- Beide Ausgänge werden 0!
  Prinzipiell nicht erlaubt, da $\neg Q$ immer die Negation von Q sein soll
- Zustand hält solange, bis R oder S wieder 0 werden

RS-Flipflop mit NAND



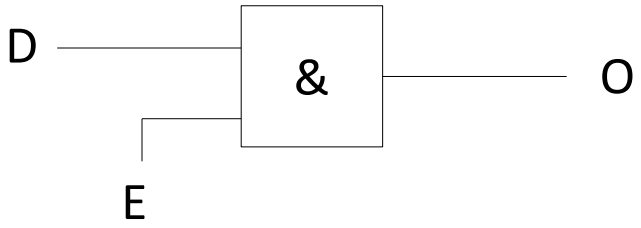
E1	E2	A1(t+1)	A2(t+1)
0	1	1	0
1	1	A1(t)	A2(t)
1	0	0	1
0	0	1	1

- *S=0* und *R=0* verbotener Zustand
- *S* und *R* sind negiert, Q und ¬Q vertauscht

Nachteil des RS-Flipflop

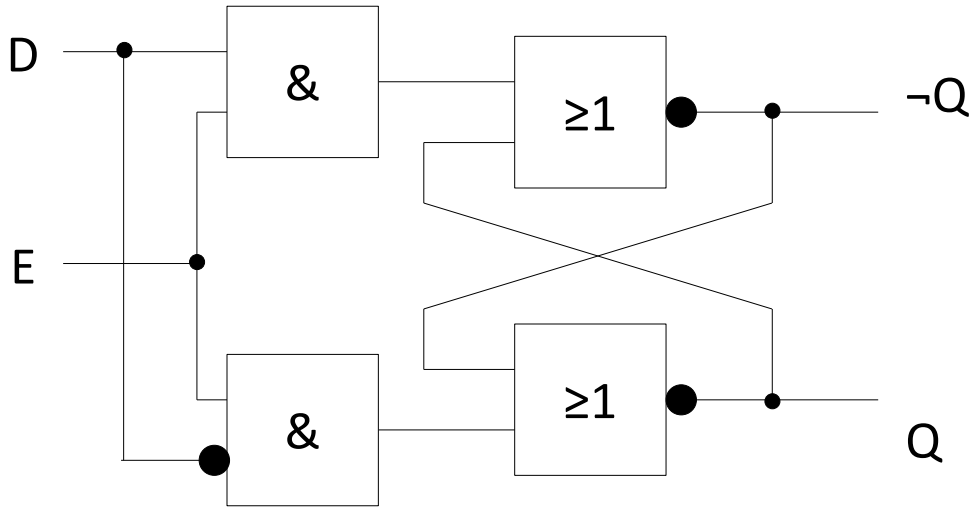
- Beim Speichern eines Wertes 0 oder 1 muss man den Wert kennen!
- Ziel: Speichern *unbekannter* Werte

UND-Gatter als „Tor“



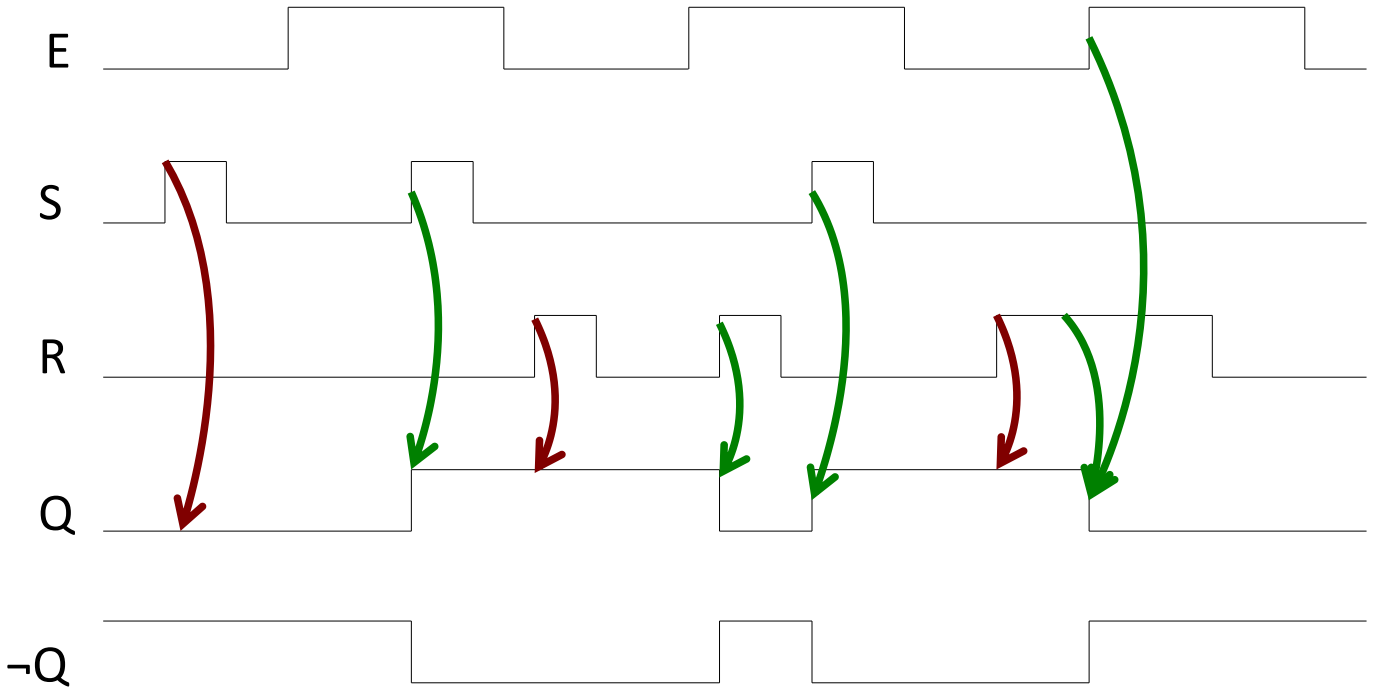
- Und-Gatter kann wie ein „Tor“ interpretiert werden
  - Ein Dateneingang
  - Ein Steuereingang
- Steuereingang=1
  - Dateneingang wird auf Ausgang durchgelassen
- Steuereingang=0
  - Datenineingang wird nicht durchgelassen

D-Latch



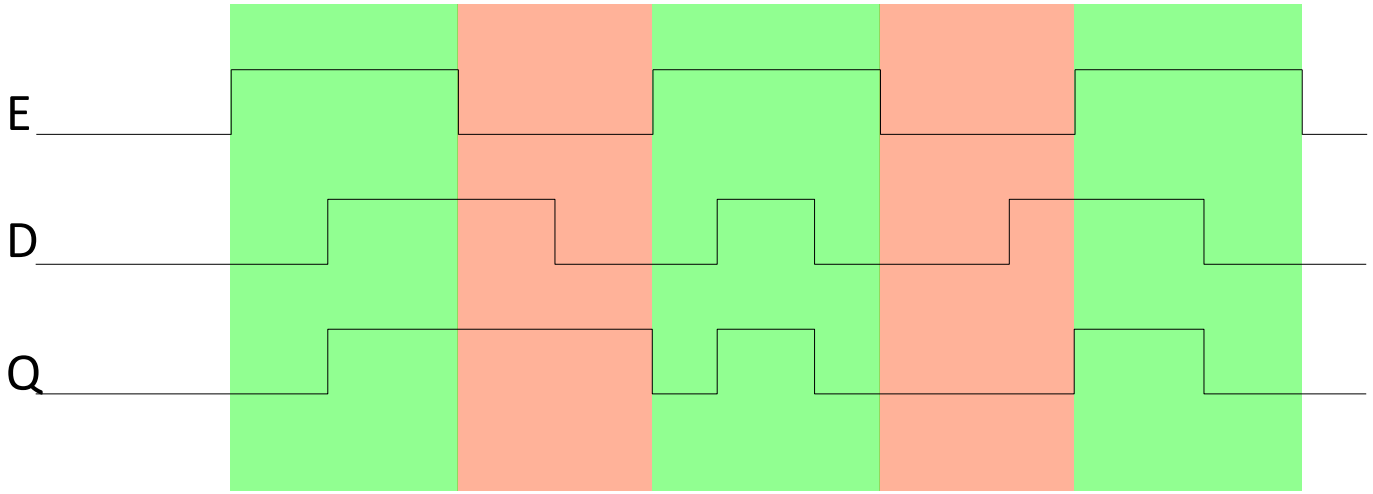
- *S* und *R* werden aus *D* berechnet
  $S = D,\ R = \neg D$
- Enable steuert die Datenübernahme

RS-Latch: Zeitdiagramm



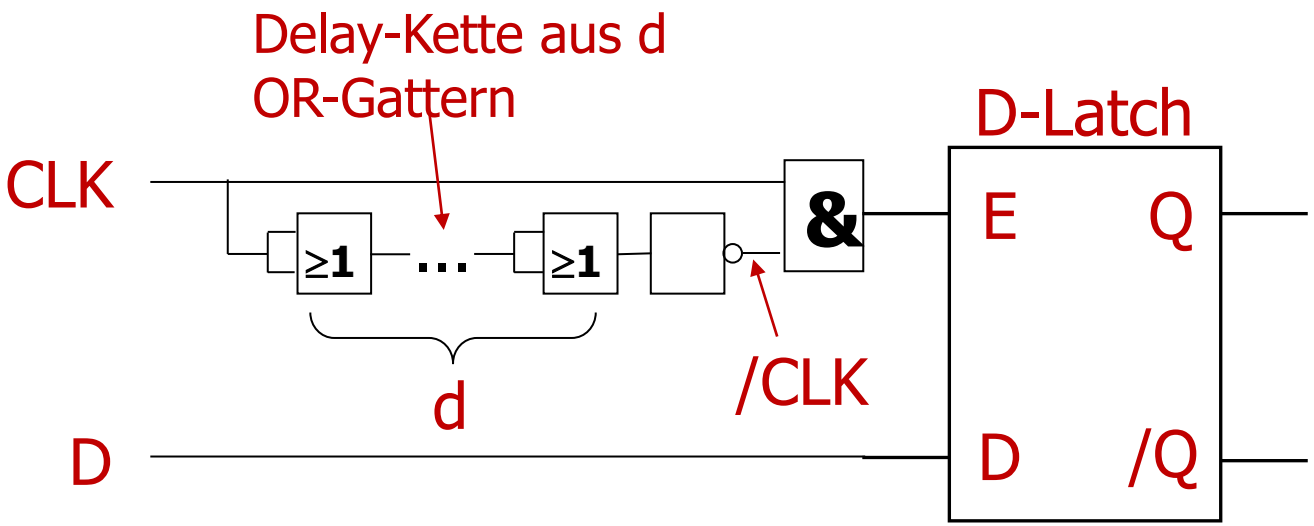
Eigenschaften eines D-Latches

- Das D-Latch heißt transparent, wenn das Schreibsignal aktiv ist.
- E muss lange genug aktiv sein, damit sich der neue Zustand im RS-FF einstellen kann.
- Das D-Latch ist pulsgesteuert (Schreibpuls E)
- Einfache 1-Bit Speicherzelle

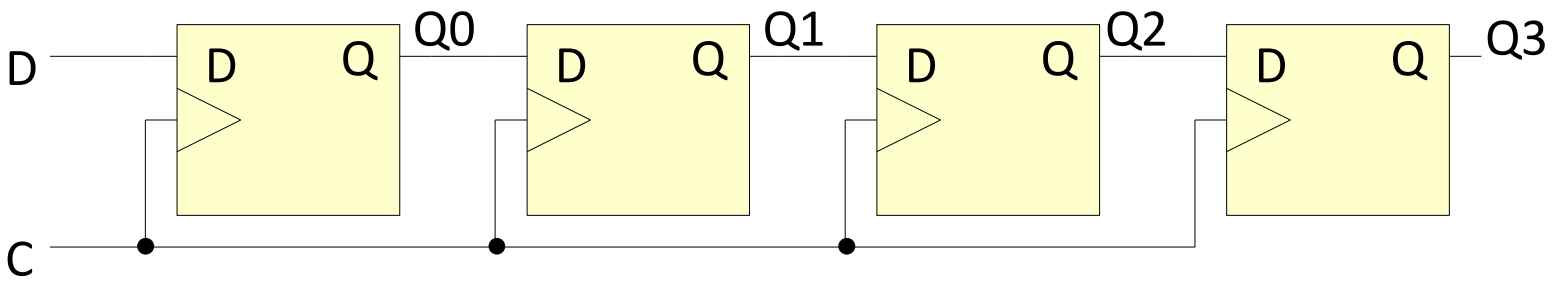


Taktflankengesteuerte D-Flipflops

- Taktflankengesteuerte Flip-Flops wie das D-Flip-Flop übernehmen Daten zu einem bestimmten Zeitpunkt (kein transparenter Modus!), nämlich bei der steigenden Flanke des sog. Clocksignals
- <u>Vorteil:</u>
  Daten müssen lediglich bei der steigenden Taktflanke stabil sein (zzgl. Setup- und Holdzeit)

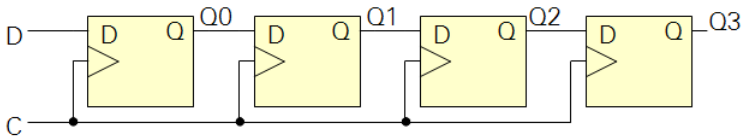


Schieberegister #1



- Alle D-FF übernehmen bei steigender Flanke
- Änderungen geschehen gleichzeitig
  Q0=D, Q1=Q0, Q2=Q1, Q3=Q2
- Information wird um eine Bitposition nach rechts geschoben

Schieberegister #2



Wert am Eingang vor dem Takt

Takt	D	Q0	Q1	Q2	Q3
		0	1	0	1
1	0	0	0	1	0
2	1	1	0	0	1
3	0	0	1	0	0
4	0	0	0	1	0
5	0	0	0	0	1

Belegung vor dem Start

Speicher

Robert Wille
robert.wille@tum.de

Chair for Design Automation
Technical University of Munich
https://www.cda.cit.tum.de/

Connect on LinkedIn

Follow @rbrtwll

Rechnersichten

- Höhere Programmiersprache

```
int a;
int b;
int c;
int d;

a = b * c;
b = (b + c) - d;

if ( a == b ) {
    cout << “Case 1”;
else {

    cout << “Case 2”;
}
...
```

- Assembly

```
#a liegt im Speicherblock $t0
#b liegt im Speicherblock $t1
#c liegt im Speicherblock $t2
#d liegt im Speicherblock $t3

mul $t0 $t1 $t2
add $t1 $t1 $t2
sub $t1 $t1 $t3
bneq $t0 $t1 ELSE
syscall … # “Case 1”
j ENDIF
ELSE:
syscall # … “Case 2”
ENDIF
…
```

- Maschinensprache

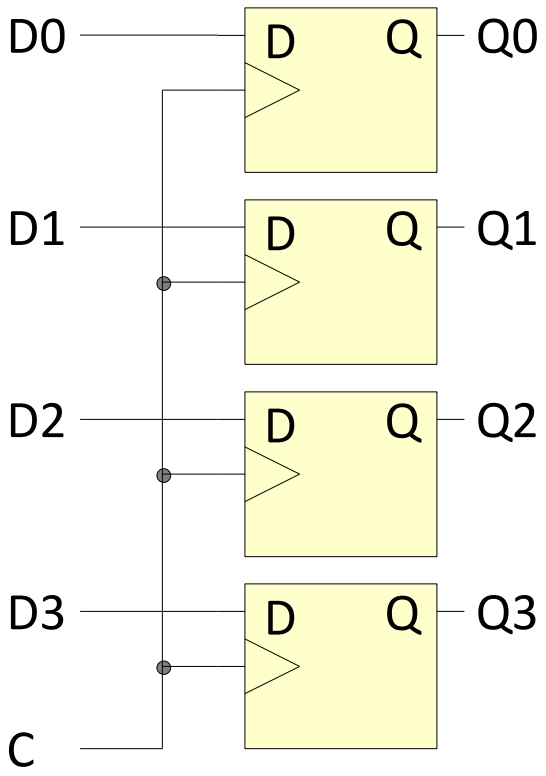
```
Daten
(im Datenspeicher in den
entsprechenden Blöcken)
Programm
(im Programmspeicher)
000100 00000 00001 00010 …
000000 00001 00001 00010 …
000010 00001 00001 00011 …
001000 00000 00001 1001001010…
100000 0101010101111011101000…
110000 00000 00001 1001001010…

100000 010101010111010 0…

…
```

Register

- Gruppierung von D-FF
- Gemeinsames Taktsignal
- Separater Ein- und Ausgang für jedes FF
- Alle FF übernehmen gleichzeitig die Eingangssignale
- Z.B. zur Speicherung von Variablen, Zwischenergebnissen, etc.

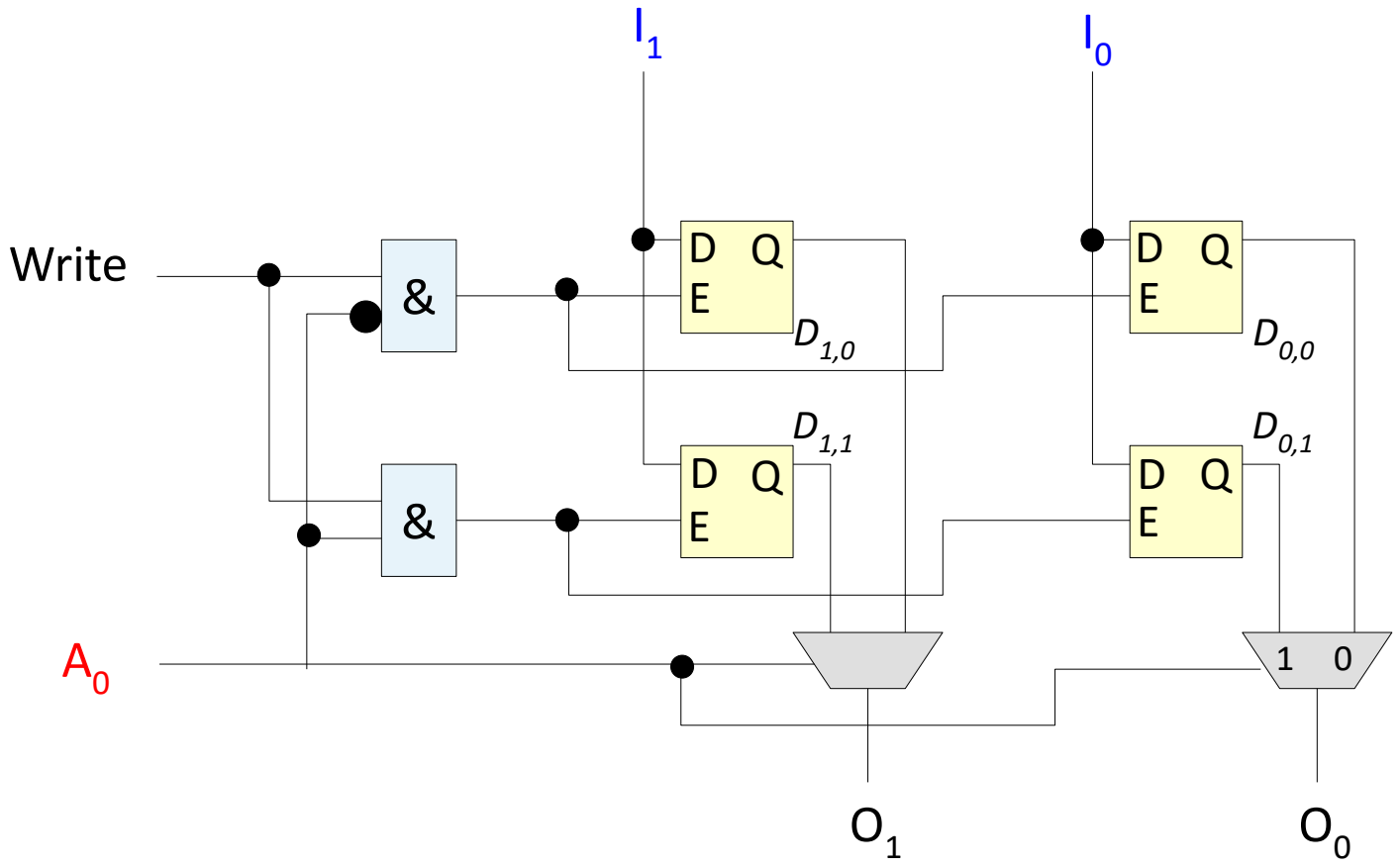


Speicher

- Bisher:
  - Speicherung von Bits in FlipFlops
  - Speicherung von Worten in Registern
- Gefordert:
  - Schaltung zur Speicherung großer Mengen von Informationen
  - Selektiver Zugriff auf Teile der gespeicherten Information
- Erster Lösungsansatz:
  - Speicherung in D-Latches
  - Auswahl durch Spezifikation eines Teilbereichs des Speichers(Adresse)

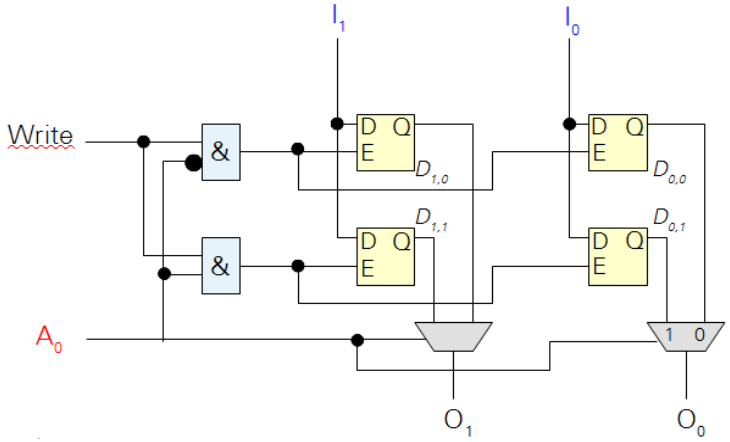
Beispiel: 2x2 Bit Speicher

- 1 Adressleitung
- 2 Dateneingänge
- 2 Datenausgänge



Beispiel: Auslesen des Speichers

- Anlegen der gewünschten Adresse
- Nach Verzögerungszeit des Multiplexers liegt Datum vor



Übertragbar auf mehrere Adressleitungen

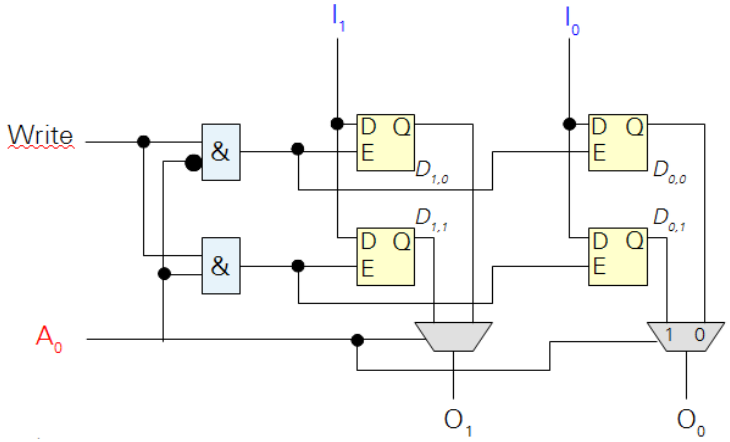
- *n* Adressleitungen ⇒ 2^n-zu-1 Multiplexer
- Manchmal auch 2^n Speicherplätze genannt

Übertragbar auf beliebige Wortbreiten

- Typische Wortbreiten: 4, 8, 16, 32, 64 Bit
- Innerhalb eines Chips meistens nur bis zu 16 Bit Wortbreite

Beispiel: Schreiben in Speicher

- Anlegen der gewünschten Adresse und des zu schreibenden Datums
- Aktivieren der Schreibleitung
- Latches übernehmen Daten
- Deaktivieren der Schreibleitung (Speicher ändert sich nicht mehr)



Übertragbar auf mehrere Adressleitungen

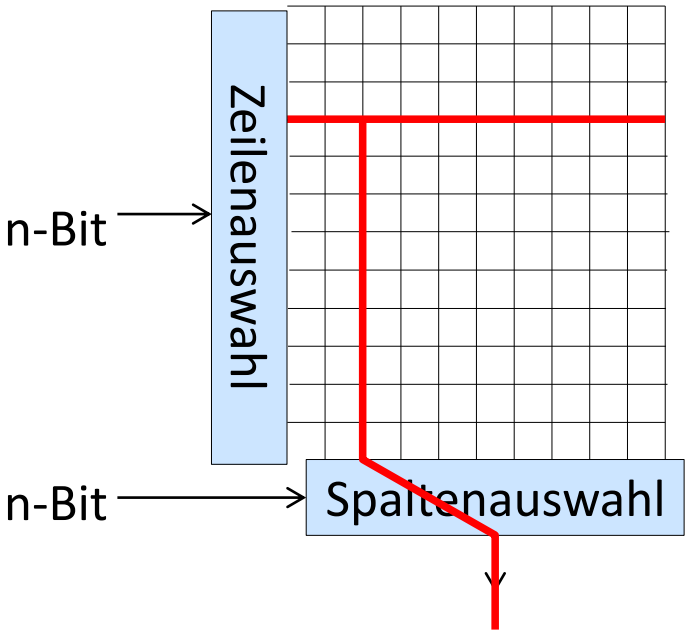
- Schreibimpuls muss jeweils für ein Wort generiert werden
- Und-Verknüpfung muss eine eindeutige Kombination der Adressbits selektieren

Reale Speicher #1

- Speicherung in D-Latch zu aufwändig
- In realen Speichern wird jeweils ein Bit gespeichert:
  - In einem Kondensator
    - Dynamic Random Access Memory (DRAM)
    - Verliert seine Ladung, muss daher regelmäßig aufgefrischt werden
  - In einer 6-Transistor-Zelle
    - Static Random Access Memory (SRAM)
    - Hält die Information beliebig lang
  - Spezielle Varianten für Festwertspeicher (ROM, EPROM, ...)

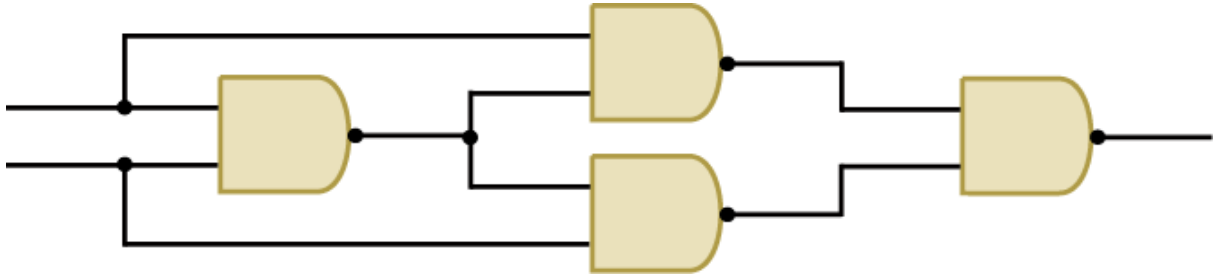
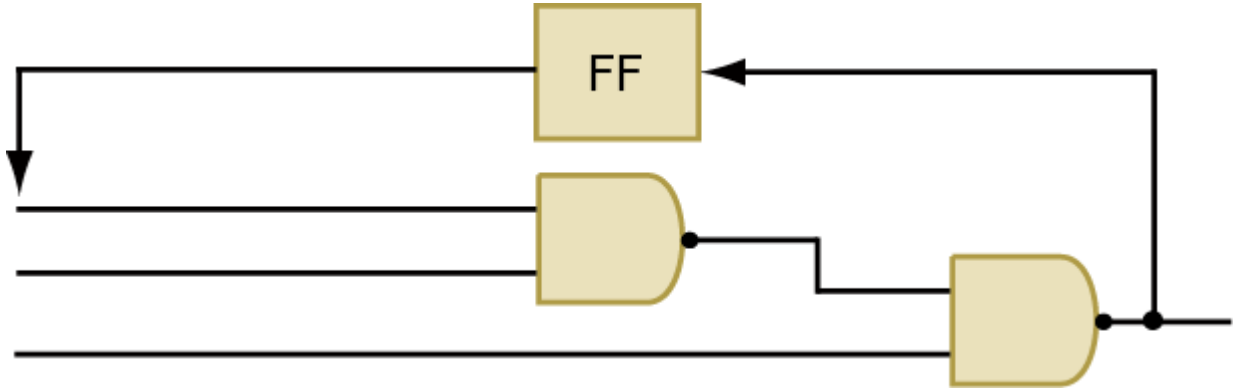
Reale Speicher #2

- Selektion der Ausgabewerte durch Multiplexer zu aufwändig
- Reale Selektion eines Speicherelementes
  - Anordnung der Speicherelemente in einer Matrix
  - Zweistufige Adressierung
  - Zunächst Aktivierung einer Zeile der Matrix
  - Dann Selektion eines Elementes der Zeile
  - Matrix meist Quadratisch
  - Beide Auswahlschaltungen verarbeiten die Hälfte der Adressbits



Sequentielle Schaltungen

- Kombinatorische Schaltungen



- Sequentielle Schaltungen